Solving the Dog Game

Live demo: charith-reddy-pareddy.github.io/dog-game

Code: github.com/Charith-Reddy-Pareddy/dog-game, reproducible with `pytest` and the commands in Section 8.

This report studies the dog game, a two-player, general-sum game with continuous actions, with two families of solver: policy-gradient self-play (REINFORCE, A2C and PPO) and Nash-Q on a discretized grid. It answers the research questions in RESEARCH_QUESTIONS.md from saved runs, and says how many runs each claim rests on.

Summary

- **Policy-gradient self-play converges on the dog game.** 74 of 75 REINFORCE runs and 25 of 25 PPO runs reached exploitability below 0.004 within 1500 episodes. PPO needs far fewer episodes: 19 of 25 runs had converged after 100, against none for REINFORCE and A2C (Section 5.2), an advantage that comes from reusing each episode (with one epoch PPO is no faster than REINFORCE), and on the walking game PPO is the least stable of the three (Sections 5.10 and 5.11).
- **Self-play never finds the solver's interior equilibrium when the houses share a coordinate.** All 30 runs ended on a boundary equilibrium, one of two mirror images, split 23 to 7 and the same for every network layout (Section 5.3).
- **Dog inertia slows training sharply.** On the harder setup 4, 5, 1 and 0 of 5 runs converged at dog weights 0, 0.3, 0.6 and 0.9 (Section 5.4).
- **Pre-training helps or hurts depending on the source.** From a setup with the same equilibrium corners, every run was under the threshold after 100 episodes; from one with flipped corners, every run was worse than starting from scratch (Section 5.5).
- **Nash-Q on a grid.** The stage-game error falls about as 1 over the fictitious-play iterations, and re-solving less often changes nothing in this game. The grid is exact from 5 x 5, the exact equilibrium is always a single action pair, and all of fictitious play's apparent mixing is noise (Sections 5.6 to 5.8).
- **Walking game.** REINFORCE and the 10-direction Nash-Q network both end near the one-shot Nash corners in all 5 seeds (Section 5.9). With A2C and PPO added, the ranking changes: A2C is the most reliable (5 of 5 seeds at the corners), while PPO is the fastest to get there but swings away again in 3 of 5 seeds. A smaller learning rate does not fix it and fewer epochs only help partly (Sections 5.10 and 5.11).
- **Open:** why self-play picks the 3-to-1 split, why mismatched pre-training hurts, the equilibrium of the dynamic games (with inertia, and walking), and a PPO comparison matched on gradient steps (Section 6).

1. Background

The dog game (also called the Battling Influencers Game) is a small, general-sum, two-player game with continuous actions. It is useful as a proving ground for multi-agent learning methods because it has a closed-form equilibrium to check against: any learned policy can be scored exactly, so "did it converge?" has a number for an answer instead of an opinion.

Two families of solver are studied here, side by side on the same environment:

- **Policy gradient**: self-play with REINFORCE, A2C and PPO, using a Gaussian policy over continuous positions.

- **Nash-Q**: a deep-Q style method on a discretized action grid, where every Bellman target requires solving a stage game for its Nash equilibrium, either approximately (fictitious play) or exactly (Lemke-Howson).

The report answers the research questions in RESEARCH_QUESTIONS.md from experiments that can be rerun from this repository. Every number below comes from a saved result file in `results/`, and every claim says how many runs it rests on.

2. The game

Two players, red and blue, each own a house at a fixed point in the unit square. Each round both players simultaneously pick a point in the square, and a shared dog moves to the weighted average `w * red + (1 - w) * blue` of the two picks. Each player is rewarded by the negative squared distance from the dog to their own house, and the discount is 0.9. The game is general-sum, not zero-sum. It has the same structure as two influencers each staking out an opinion to pull a shared audience toward them. Because the domain is bounded, a player who wants the dog exactly on their house has to overshoot: pick further out than their own house to counteract the other player's pull.

2.1 Why the equilibrium sits on the boundary

Substituting each player's unconstrained best response into the other's shows there is no interior fixed point unless the houses share a coordinate. The unconstrained recursion pushes the actions further out on every iteration. The bounded domain turns this into a genuine equilibrium: both players are pushed to the boundary and exaggerate as much as the square allows. Iterated best response (`doggame/nash.py`) reaches this fixed point in two rounds, and it is used throughout as the ground truth.

2.2 Uniqueness

The equilibrium is unique when the two houses differ in every coordinate. Each coordinate the houses share becomes a degenerate direction: any pair of actions on that axis that averages back to the shared value is equally good, so the equilibrium set is a continuum. Identical houses are the extreme case, where no exaggeration is needed at all.

2.3 Two variants

- **Dog inertia.** `make_inertial_transition(w0)` gives the dog momentum: `dog' = w0 * dog + (1 - w0) * (w * red + (1 - w) * blue)`. Holding both actions fixed, the dog still ends at the same steady state as without inertia; it only gets there more slowly, at rate `w0` per round.
- **The walking game (the angle-radius version).** In the main game a player jumps straight to any point it likes. In the walking game each player has a position, starting on its own house, and each round picks a direction (an angle) and a step length of up to 0.2, then walks that way from where it stands, stopping at the edge of the square. The dog moves to the average of the two players' new positions, and each player is rewarded by the negative squared distance from the dog to its own house, as before. The state is therefore the dog's position plus both players' positions, and an action is a polar pair (angle, step length) instead of a point. The angle comes from a von Mises distribution and the step fraction from a Beta distribution, so the angle wraps cleanly and nothing is clipped. This is one reading of "angle-radius action", described in ASSUMPTIONS.md.

3. Methods

3.1 Policy gradient

A Gaussian policy (`doggame/agents.py`) gives both players' action means through one of three network layouts, plus a learned per-player log standard deviation (initialized at -0.5, a standard deviation of about 0.61). Training is self-play, one episode of 20 rounds per update, with Adam at learning rate 0.01 through PyTorch's optimizer. Three algorithms share this setup:

- **REINFORCE** (`doggame/train.py`): the log-probability of each action weighted by the discounted return minus the episode's mean return.
- **A2C** (`doggame/train_ac.py`): the same, but the baseline is a learned value function of the dog's position, one per player.
- **PPO** (`doggame/train_ac.py`): A2C with each episode reused for 4 epochs and the probability ratio clipped at 0.2.

Three network layouts sit behind one interface (`doggame/networks.py`): **separate** (two independent networks), **shared** (one trunk with two heads) and **partial** (a shared trunk feeding two separate per-player branches).

3.2 Nash-Q on a discretized grid

`doggame/discretize.py` turns the square into an N x N grid of actions and builds both players' payoff matrices with the same transition and reward functions as the continuous game. `doggame/dqn.py` iterates the Nash-Q backup `Q = R + discount * NashV(Q)` to a fixed point. Because the dog game's transition does not depend on the incoming state, this collapses to iterating on one pair of payoff matrices, and the fixed point has a closed form: the discounted value equals the one-shot Nash value divided by `1 - discount`. That shortcut is specific to this game. The stage game is solved either by fictitious play (`doggame/fictitious_play.py`, approximate) or exactly by NashPy's Lemke-Howson algorithm (`doggame/exact_nash.py`). The exact solver is hardened: every answer is checked to be an equilibrium, retried from another starting label if not, and cut off after a fixed number of pivots, so a result never depends on how fast the machine is.

3.3 The walking game

`doggame/polar_agent.py` trains the walking game's policy with REINFORCE, A2C or PPO (the actor-critic loop of `doggame/train_ac.py` is shared with the main game). `doggame/polar_dqn.py` is a deep Nash-Q for the same game: 10 compass directions, one network giving both players' 10 x 10 Q-values, a target network, a replay buffer and an exact Nash solve at every target.

4. Verification

A solved or learned policy is checked directly instead of trusting how it was produced (`doggame/verify.py`):

- **Exploitability**: for a continuous action pair, search a 101 x 101 grid for the largest gain either player could get by deviating. It is 0 at an equilibrium.
- **Indifference gap**: for a discrete mixed strategy, every action played with positive probability should have equal value, and no unplayed action should do better. The check uses a probability floor (`tol`, 0.001) to decide what counts as played.

A defect found and fixed. On a fine enough action grid, a uniform mixed strategy spreads every probability below the floor, so the check selected no actions and silently returned infinity instead of a number. It now falls back to the most probable actions when none clear the floor. This is covered by a regression test, and it

matters for the resolution question below.

5. Experiments

Every experiment is a function in [`doggame/dog_game_studies.py`](doggame/dog_game_studies.py) or [`doggame/dog_game_study.py`](doggame/dog_game_study.py). Raw results are in `results/dog_game_runs.jsonl` and `results/dog_game_studies.jsonl`, one line per run, and the tables below are produced from them by `doggame/dog_game_summary.py`. A run counts as **converged** when its exploitability is below 0.004. That threshold is my choice, so each table also shows the median.

5.1 Does policy-gradient self-play converge?

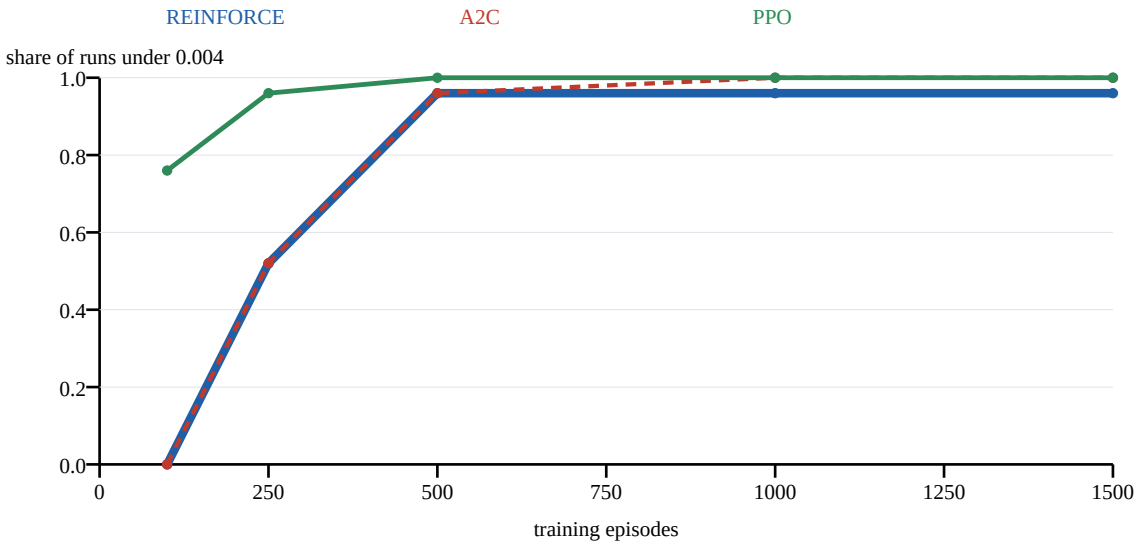
REINFORCE self-play, 1500 episodes, five house setups and three network layouts, five seeds each (75 runs). Each cell is the number of runs under 0.004 with the median exploitability.

House setup	separate	shared	partial
(0.9, 0.2) / (0.1, 0.8), w = 0.5	5 of 5 (median 0.0002)	5 of 5 (median 0.0002)	5 of 5 (median 0.0004)
(0.8, 0.8) / (0.2, 0.3), w = 0.3	4 of 5 (median 0.0007)	5 of 5 (median 0.0006)	5 of 5 (median 0.0008)
(0.7, 0.1) / (0.6, 0.9), w = 0.7	5 of 5 (median 0.0003)	5 of 5 (median 0.0003)	5 of 5 (median 0.0008)
(0.95, 0.5) / (0.05, 0.5), w = 0.5	5 of 5 (median 0.0001)	5 of 5 (median 0.0001)	5 of 5 (median 0.0002)
(0.6, 0.6) / (0.4, 0.4), w = 0.5	5 of 5 (median 0.0001)	5 of 5 (median 0.0001)	5 of 5 (median 0.0002)

- 74 of 75 runs converged. The one miss is the `separate` layout on setup 2, seed 0, at 0.18. The other four seeds of that setup converge. An earlier single-seed note said that training the same run 3000 episodes longer brought it to 0.004; that was not rerun here.
- The three layouts do not separate: `separate` 24 of 25, `shared` 25 of 25, `partial` 25 of 25. The one miss belongs to `separate`, but a single run cannot rank architectures.
- The threshold matters: the closest passing run is at 0.0039 (`shared`, setup 2, seed 0), so the count depends on where the line sits.

5.2 REINFORCE, A2C and PPO

The same five setups and seeds, the `separate` layout, 1500 episodes, scored at five points during training (25 runs per algorithm). The chart shows the share of runs under 0.004 by episodes trained.



Algorithm	100 episodes	250 episodes	500 episodes	1000 episodes	1500 episodes	Median at the end
REINFORCE	0 of 25	13 of 25	24 of 25	24 of 25	24 of 25	0.0002
A2C	0 of 25	13 of 25	24 of 25	25 of 25	25 of 25	0.0001
PPO	19 of 25	24 of 25	25 of 25	25 of 25	25 of 25	0.0000

- **PPO is much faster per episode.** 19 of 25 PPO runs converged after 100 episodes, against none for REINFORCE and A2C, and all 25 converged by 500.
- **A2C is not faster than REINFORCE** for the first 500 episodes, but it converged on all 25 runs by 1000 episodes, including the slow run that REINFORCE left unconverged.
- **The comparison is matched on episodes, not on gradient steps.** PPO takes four gradient steps on each episode and REINFORCE takes one. Section 5.11 reruns PPO with fewer epochs to separate the two.

5.3 Which equilibrium does self-play find?

In the setup whose houses share a y-coordinate, the equilibrium set has a continuum of members. Iterated best response (`doggame/nash.py`) returns the interior point with both players at y = 0.5. Thirty REINFORCE runs (ten seeds per layout) ended here:

Layout	red top, blue bottom	red bottom, blue top	both at y = 0.5	other	Median exploitability
separate	8 of 10	2 of 10	0 of 10	0 of 10	0.00009
shared	7 of 10	3 of 10	0 of 10	0 of 10	0.00008
partial	8 of 10	2 of 10	0 of 10	0 of 10	0.00018

- **Never the interior point.** All 30 runs ended with one player at the top edge and the other at the bottom edge. Both are exact equilibria (exploitability below 0.0002), as is the interior point.
- **Two mirror equilibria, split about 3 to 1.** 23 runs ended with red at the top and 7 with red at the bottom. The layouts do not differ (8, 7 and 8 of 10).
- **The exact solver agrees with the learners.** Lemke-Howson on the discretized game also returns a boundary equilibrium for this setup, at every grid size (Section 5.7).

- **Why is not tested.** One hypothesis is that the policy's initial standard deviation (about 0.61, large for a unit square) pushes early samples onto the boundary, where clipping removes any gradient along the degenerate axis. Nothing here tests that.

5.4 Does dog inertia change the outcome?

REINFORCE, five seeds, 1500 episodes, with the dog's own weight `w0` set to 0, 0.3, 0.6 or 0.9, on the easiest setup and on the setup that was slowest in Section 5.1. Policies are scored against the one-shot game.

House setup	w0 = 0.0	w0 = 0.3	w0 = 0.6	w0 = 0.9
(0.9, 0.2) / (0.1, 0.8), w = 0.5	5 of 5 (median 0.0002)	5 of 5 (median 0.0002)	5 of 5 (median 0.0002)	5 of 5 (median 0.0006)
(0.8, 0.8) / (0.2, 0.3), w = 0.3	4 of 5 (median 0.0007)	5 of 5 (median 0.0016)	1 of 5 (median 0.0102)	0 of 5 (median 0.0295)

- **Inertia slows training a great deal on the hard setup.** 4, 5, 1 and 0 of 5 runs converged within 1500 episodes at `w0` = 0, 0.3, 0.6 and 0.9. On the easy setup every run converged at every `w0`, with a slightly higher median at 0.9.
- **The unconverged runs look unfinished, not different.** Their median distance to the one-shot equilibrium stays small (0.0016, 0.0011, 0.0072 and 0.0176 over all runs at each `w0`), but the worst runs at `w0` of 0.6 and 0.9 are about 1.0 away, meaning one player is still far from its corner. Whether more episodes fix them was not tested.
- **What this does not say.** Policies are scored against the one-shot equilibrium, which is the steady state of the dynamic game. Whether the equilibrium of the full game with inertia is the same is not solved here.

5.5 Does pre-training help?

REINFORCE on setup 2, the slowest one, five seeds. "Pre-trained" means 1500 self-play episodes on another setup first, then training on setup 2. Equilibrium corners from the exact solver: setup 1 has red at bottom-right and blue at top-left; setups 2 and 5 have red at top-right and blue at bottom-left.

Starting point	after 100 episodes	after 300 episodes
from scratch	0 of 5 (median 0.0599)	2 of 5 (median 0.0048)
pre-trained on setup 1	0 of 5 (median 0.1599)	0 of 5 (median 0.2289)
pre-trained on setup 5	5 of 5 (median 0.0002)	4 of 5 (median 0.0002)

- **It depends on the source.** Pre-training on setup 5, which has the same corners as setup 2, put all 5 runs under 0.004 after 100 episodes, where training from scratch had none.
- **It can hurt.** Pre-training on setup 1, whose corners are flipped, left every run worse than scratch after 300 episodes (median 0.23 against 0.0048).
- **One target setup and five seeds.** The mechanism is not tested; a plausible one is that the network has to un-learn a boundary direction it already committed to.

5.6 How precisely must the stage game be solved?

Nash-Q on the 9 x 9 grid of setup 1, discount 0.9. Fictitious play replaces the exact solver with a given number of iterations, and the stage game is re-solved every 1, 5 or 25 outer iterations. Each cell is the median error of the fixed point's Q-values against the exact solution, and the median gap (the most either player could gain by switching to a single action) of the final strategy. Five seeds per cell.

Fictitious-play iterations	re-solve every 1: Q error / gap	re-solve every 5: Q error / gap	re-solve every 25: Q error / gap
10	0.3612 / 0.0583	0.3612 / 0.0583	0.3612 / 0.0583
30	0.1311 / 0.0215	0.1311 / 0.0215	0.1311 / 0.0215
100	0.0406 / 0.0067	0.0406 / 0.0067	0.0406 / 0.0067
300	0.0136 / 0.0023	0.0136 / 0.0023	0.0136 / 0.0023
1000	0.0041 / 0.0007	0.0041 / 0.0007	0.0041 / 0.0007
3000	0.0014 / 0.0002	0.0014 / 0.0002	0.0014 / 0.0002

- **The error falls about as 1 over the number of iterations.** It is 0.36 at 10 iterations and 0.0014 at 3000, so a given Q error can be bought with a predictable number of iterations.
- **Re-solving less often changes nothing here.** Every column is the same. In this game the transition does not depend on the state, so each Q matrix is the payoff matrix plus a constant, and adding a constant never moves an equilibrium. This will not hold in a game where the continuation value varies by state.

5.7 Does grid resolution matter?

The exact (Lemke-Howson) solution of the discretized stage game on five grid sizes, for all five setups (25 games).

Grid	Actions per player	Matrix entries	Setups with a unique equilibrium found (of 4)	Shared-y setup: distance to the solver's equilibrium	Equilibrium is one action pair	Seconds (median)
3 x 3	9	81	2	1.00	yes	0.003
5 x 5	25	625	4	1.00	yes	0.031
9 x 9	81	6,561	4	1.00	yes	0.356
13 x 13	169	28,561	4	1.00	yes	2.017
17 x 17	289	83,521	4	1.00	yes	116.625

- **Quality stops improving at 5 x 5.** The equilibrium sits on the boundary, and every grid contains the corners, so a finer grid finds nothing new. A 3 x 3 grid is too coarse: for some setups it picks a different corner.
- **Cost grows steeply.** The payoff matrices have the square of the number of actions as entries, and the exact solve time grows at least as fast. These timings come from a heavily loaded machine, so only the growth matters, not the absolute seconds.
- **Ties make it far slower than the median shows.** On the setup with the most tied payoffs (houses on a diagonal), the exact solve took 17 seconds at 9 x 9, 338 seconds at 13 x 13 and about 43 minutes at 17 x 17, because most starting labels end in an error and are retried from another label. The other setups took seconds at the same sizes.
- **The equilibrium is always one action pair.** In all 25 games the exact solution has support 1 x 1.
- **The shared-y setup returns a boundary equilibrium** at every size, 1.0 away from the interior point the best-response solver reports. Both are valid (Section 5.3).

5.8 How much of fictitious play's mixing is noise?

Fictitious play on the same game for a range of iteration counts, five seeds each, scored with the indifference check at its default floor of 0.001.

Fictitious-play iterations	Indifference spread	Actions counted as played	Weight on the leftover first pick
30	0.6415	2	0.03226
100	0.6496	2	0.00990
300	0.6519	2	0.00332
1000	0.0000	1	0.00100
3000	0.0000	1	0.00033
10000	0.0000	1	0.00010
exact (Lemke-Howson)	0.0000	1	0.00000

- **The exact equilibrium is one action pair, so none of the mixing is real.** What fictitious play shows is the weight of its first random pick, which decays as 1 over the number of iterations plus one.
- **The check hides it only once it falls below the floor.** At 300 iterations or fewer the leftover weight is above 0.001, so a second action counts as played and the indifference spread is about 0.65. From 1000 iterations the weight is at or below 0.001 and the spread is exactly 0.
- **A structural fix is available.** Dropping the first pick's weight from the running average would remove the noise instead of filtering it. It is not implemented.

5.9 The walking game

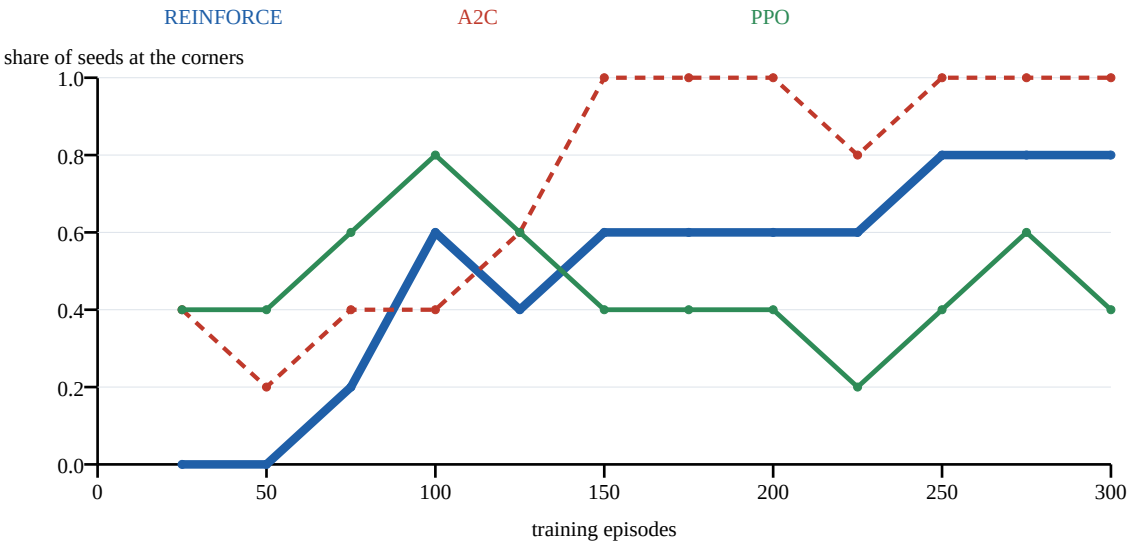
The angle-radius version, on one house setup (red at (0.88, 0.12), blue at (0.3, 0.65), w = 0.5, maximum step 0.2). The score is the sum of both players' distances from the one-shot Nash corners at the end of a 15-step game, averaged over 10 sampled games, over five seeds. REINFORCE trains 300 episodes and the Nash-Q network 60.

Method	Distance before training	Distance after training	Seeds that improved
REINFORCE (angle and step)	1.49	0.04	5 of 5
Nash-Q (10 directions)	1.95	0.01	5 of 5

- **Both methods walk to the corners.** The saved paths for five seeds are on the live site with a step slider and a play button.
- **What this does not show.** The one-shot corners are the equilibrium of the one-shot game. The walking game itself has not been solved, so this shows the players heading to the one-shot corners, not that those corners are an equilibrium of the walking game. One untrained REINFORCE policy (seed 4) started near the corners anyway.

5.10 REINFORCE, A2C and PPO on the walking game

The same three algorithms on the walking game (Section 5.9's house setup), five seeds each (15 runs), 300 episodes, with the same 10-game score measured every 25 episodes. The chart shows the share of the five seeds whose summed distance from the Nash corners is below 0.1 (my threshold). In the table, "At N" is the mean distance after N episodes of training. The paths swing, so it also reports how many of the later checkpoints (from 150 episodes on, 7 per seed) are at the corners.



Algorithm	Before training	At 50	At 100	At 200	At 300	Late checkpoints at the corners (from 150)	Seeds at the corners at the end
REINFORCE	1.49	0.696	0.341	0.130	0.055	24 of 35	4 of 5
A2C	1.49	0.731	0.199	0.032	0.011	34 of 35	5 of 5
PPO	1.49	0.263	0.127	0.500	0.371	14 of 35	2 of 5

- **The ranking is not the main game's.** On the main game PPO was the fastest and most reliable. On the walking game **A2C is the most reliable**: all 5 seeds end at the corners and 34 of its 35 later checkpoints are under 0.1.
- **REINFORCE is slower and more variable.** It reaches the corners later (mean distance 0.34 at 100 episodes against 0.20 for A2C) and spends 24 of 35 later checkpoints there; one seed stays near 0.2.
- **PPO is the fastest to get there and the worst at staying.** It has the lowest distance at 50 and 100 episodes, but seeds 1, 2 and 4 leave the corners again later (their distance climbs back to about 1.0, meaning a player has left its corner), and it spends only 14 of 35 later checkpoints at the corners.
- **A single end-of-training number would mislead.** A first run of this study, scored only at 50, 100, 200 and 300 episodes, gave the same order but different per-seed outcomes (PPO seed 3 ended at 1.4 there and at 0.0 here), because evaluating the policy draws random numbers and so changes the training run. The paths are sensitive, which is why this study records every 25 episodes and reports the share of checkpoints.
- **Why PPO is unstable here is not tested.** Possible causes are four epochs on a 15-step batch, a learning rate of 0.01 and advantage normalization over so few samples. Whether a smaller learning rate or fewer epochs fixes it was not run.

5.11 Where does PPO's speed come from, and can it be made stable?

Two follow-ups to Sections 5.2 and 5.10, 65 more runs.

Main game, matched on gradient steps. PPO takes one gradient step per epoch on each episode, and REINFORCE and A2C take one in total. The same five setups and seeds, the `separate` layout, with PPO at 1 and 2 epochs next to the default 4:

Algorithm	Gradient steps per episode	100 episodes	250 episodes	500 episodes	1000 episodes	1500 episodes	Median at the end
REINFORCE	1	0 of 25	13 of 25	24 of 25	24 of 25	24 of 25	0.0002
A2C	1	0 of 25	13 of 25	24 of 25	25 of 25	25 of 25	0.0001
PPO, 1 epoch	1	1 of 25	12 of 25	20 of 25	24 of 25	25 of 25	0.0002
PPO, 2 epochs	2	9 of 25	22 of 25	23 of 25	25 of 25	25 of 25	0.0000
PPO, 4 epochs (default)	4	19 of 25	24 of 25	25 of 25	25 of 25	25 of 25	0.0000

- **The speed-up comes from reusing each episode.** After 100 episodes 1, 9 and 19 of 25 runs had converged with 1, 2 and 4 epochs. With one epoch PPO is no faster than REINFORCE (20 of 25 against 24 of 25 after 500 episodes), so the clipped objective and the learned baseline are not what makes it fast.
- **Per gradient step it is not clearly ahead.** PPO with 4 epochs after 100 episodes has taken 400 steps and has 19 of 25 runs converged. REINFORCE after 250 episodes (250 steps) has 13 and after 500 (500 steps) has 24. So PPO is more efficient in samples, because each episode is used several times, but not clearly in steps.

Walking game, a gentler PPO. PPO with one epoch and/or a learning rate of 0.003 instead of 0.01, five seeds each, scored as in Section 5.10 (300 episodes):

Setting	Before training	At 50	At 100	At 200	At 300	Late checkpoints at the corners (from 150)	Seeds at the corners at the end
A2C (reference)	1.49	0.731	0.199	0.032	0.011	34 of 35	5 of 5
PPO, lr 0.01, 4 epochs (default)	1.49	0.263	0.127	0.500	0.371	14 of 35	2 of 5
PPO, lr 0.01, 1 epoch	1.49	0.595	0.479	0.249	0.123	21 of 35	4 of 5
PPO, lr 0.003, 4 epochs	1.49	0.477	0.361	0.448	0.362	15 of 35	1 of 5
PPO, lr 0.003, 1 epoch	1.49	0.836	0.641	0.470	0.322	8 of 35	2 of 5

- **Fewer epochs helps but does not fix it.** One epoch at learning rate 0.01 ends with 4 of 5 seeds at the corners and 21 of 35 later checkpoints there, against 2 of 5 and 14 of 35 for the default. It is slower early (mean distance 0.60 after 50 episodes, against 0.26).
- **A smaller learning rate does not help.** It gives 1 of 5 seeds at 4 epochs and 2 of 5 at one epoch. Several of those runs were still far from the corners at 300 episodes, so a longer budget might change this; that was not run.
- **None matches A2C**, which has 5 of 5 seeds and 34 of 35 later checkpoints at the corners. The instability is therefore not just a step-size problem, and reusing each episode seems to contribute, but why is not tested.
- Five seeds per setting and a 300-episode horizon, so small differences between settings should not be read as a ranking.

6. Research questions and answers

The statuses are the same as in RESEARCH_QUESTIONS.md.

Q1. Can self-play policy gradient converge to a Nash equilibrium in a general-sum game? *Answered for the dog game; open in general.* 74 of 75 REINFORCE runs and 25 of 25 PPO runs converged (Sections 5.1, 5.2). Independent policy gradient can cycle in general-sum games, so this should not be read as a general result. The dog game is favorable: each best response is an affine function of the other player's action, and the bounded domain turns the recursion into a contraction toward the boundary.

Q2. Does the number of episodes depend on the configuration? *Answered.* Yes, and on the algorithm. At 250 episodes REINFORCE had 13 of 25 runs converged overall, but 0 of 5 on setup 3 and 5 of 5 on setup 5. Any episode count needs the setup and algorithm it was measured on.

Q3. Does pre-training help? *Answered for one target.* Section 5.5: it helps when the source has the same equilibrium corners and hurts when it does not.

Q4. Which equilibrium does self-play find? *Answered for the shared-y setup.* Section 5.3: never the interior point, always one of two mirror boundary equilibria, about 3 to 1, regardless of layout.

Q5. How precisely must the stage game be solved? *Answered for the dog game.* Section 5.6: the error falls about as 1 over the iterations, and re-solving less often does not matter when the transition is state-independent. **Q5b** (the literature) is not surveyed here.

Q6. How does resolution trade off against quality? *Answered.* Section 5.7: quality is exact from 5 x 5, cost grows steeply, and the equilibrium is always a single action pair. A function approximator becomes attractive when the action set grows past a few hundred actions, which is where the exact solve already takes seconds to minutes.

Q7. Which network architecture? *Partly answered.* All three converge (24, 25 and 25 of 25). Cross-play between independently trained layouts is untested.

Q8. When is the equilibrium unique? *Answered.* When the houses differ in every coordinate; each shared coordinate opens a continuum (Section 2.2, confirmed by Section 5.3).

Q9. How is an equilibrium verified? *Answered.* Exploitability for continuous actions and the indifference check for mixed strategies (Section 4).

Q10. How much non-clean mixing is real? *Answered for the dog game.* None of it: the exact equilibrium is one action pair, and the mixing fictitious play shows is the leftover of its first random pick (Section 5.8).

Q11. What transfers to harder games? *Open.* The state-independent transition (which collapses Nash-Q to one stage game), the unique equilibrium and the one-shot payoffs are dog-game specific. The solver hardening and the verification checks are game-agnostic.

Q12. Does inertia change anything besides how long it takes? *Partly answered.* The steady state is unchanged, but policy-gradient training slows sharply on the hard setup (Section 5.4). Whether the equilibrium of the dynamic game changes is untested (Q19).

Q13-Q15. Q13 (vertex Nash for multi-state games) is open because the dog game has one state. Q14 (LLM play and inverse reward recovery) is a separate research thread. Q15 (disk-shaped action spaces) is partly addressed by the walking version's angle and step length.

Q16. Do A2C and PPO beat REINFORCE? *Answered.* On the main game PPO is much faster per episode and A2C is not faster than REINFORCE (Section 5.2). With one epoch, matching REINFORCE on gradient steps, PPO is no faster than REINFORCE, so its speed comes from reusing each episode (Section 5.11, Q24). On the walking game the order changes: A2C is the most reliable and PPO is fast but unstable (Section 5.10, Q26).

Q17-Q20. Q17 (why the outcome is 3 to 1 and never interior) and Q20 (why mismatched pre-training hurts) have measured patterns but untested mechanisms. Q18 is answered (Section 5.4). Q19 (the equilibrium of the dynamic game with inertia) is open.

Q21. Does re-solving less often change the fixed point? *Answered.* No, in this game (Section 5.6).

Q22. Is the exact equilibrium always one action pair? *Answered.* Yes, in all 25 games (Section 5.7).

Q23 and Q25. Whether the one-shot corners are an equilibrium of the walking game, and the effect of step length and discount, are open.

Q24. Does PPO stay ahead when matched on gradient steps? *Answered for the main game.* No clear lead per step: with one epoch PPO is no faster than REINFORCE, and with 4 epochs it is in line with REINFORCE given the same number of steps (Section 5.11).

Q26. Do A2C and PPO beat REINFORCE on the walking game? *Answered for one house setup (15 runs).* A2C is the most reliable, REINFORCE is slower, and PPO is the fastest but does not stay at the corners (Section 5.10).

Q27-Q29. Why PPO is unstable on the walking game is open on the mechanism: a smaller learning rate does not help and fewer epochs help partly (Q28, Section 5.11). Whether the slower settings settle with more than 300 episodes (Q29) is open.

7. Limits

- Most results rest on five seeds, and the equilibrium-selection study on ten per layout. Percentages are counts out of those, not estimates with error bars.
- The convergence threshold (0.004) and the pre-training and inertia setups are my choices. Training settings (learning rate 0.01, one episode per update) were not tuned for each algorithm.
- PPO and A2C are compared mainly on episodes. Section 5.11 adds a comparison on gradient steps for the main game. Wall-clock time was not compared.
- Policies trained with inertia are scored against the one-shot equilibrium. The walking game is scored against the one-shot corners. Neither is the equilibrium of its own dynamic game.
- The pre-training study has a single target setup. The walking study has a single house setup.
- The walking-game scores use 10 sampled games per checkpoint and five seeds, and the paths are noisy, so small differences between algorithms should not be read as a ranking.
- Timings in the resolution table were taken on a heavily loaded machine and show growth, not absolute cost.
- The Gaussian policy clips actions to the square before taking the log-probability, which biases the gradient slightly. Every algorithm shares this.

8. Reproducing

```
python3 -m pytest # the test suite python3 -m doggame.experiments # the original convergence study python3 -m doggame.dog_game_study # five-seed convergence and walking runs python3 -m doggame.dog_game_studies # the nine studies in Section 5 (resumable, shardable) python3 -m doggame.dog_game_summary # print every table as Markdown python3 -m doggame.research_report_pdf # build research_report.pdf from this file
```

The studies write one line per finished run, so a stopped run keeps its results and a rerun skips what is done.

The code, results and this report are released under the MIT license.